100 XP

# Create volumes

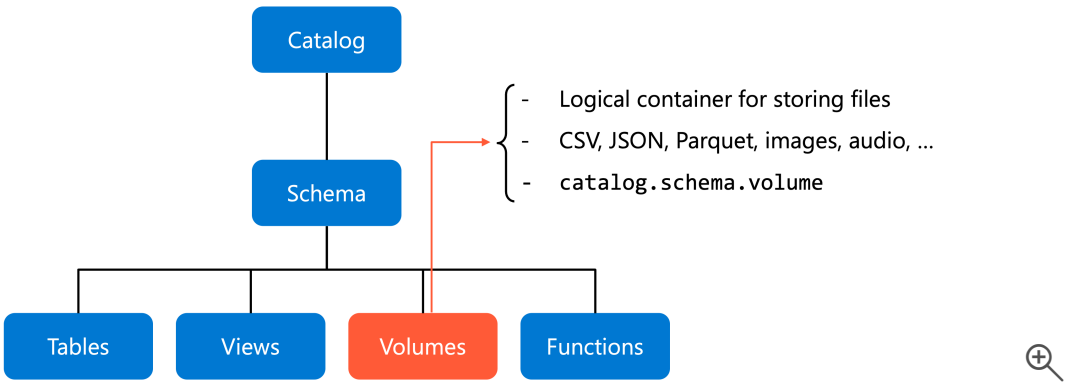
8 minutes

Your data engineering team needs to store machine learning models, CSV files for ingestion, and image files for analytics workloads. You already organized your data using catalogs and schemas, but you need a governed way to manage these nontabular files in cloud object storage. **Unity Catalog volumes** provide the solution by extending governance to files while maintaining the same security model you use for tables.

## What are volumes?

A **volume** is a Unity Catalog object that represents a logical container for files stored in cloud object storage. Think of it as the final piece in organizing your data house—after building the foundation with catalogs and organizing the interior with schemas, volumes provide the space to store your raw files.

Volumes sit at the third level of Unity Catalog's namespace structure: `catalog.schema.volume`. Unlike tables that govern structured data, volumes govern files of any format—CSV, JSON, Parquet, images, audio files, or machine learning artifacts.



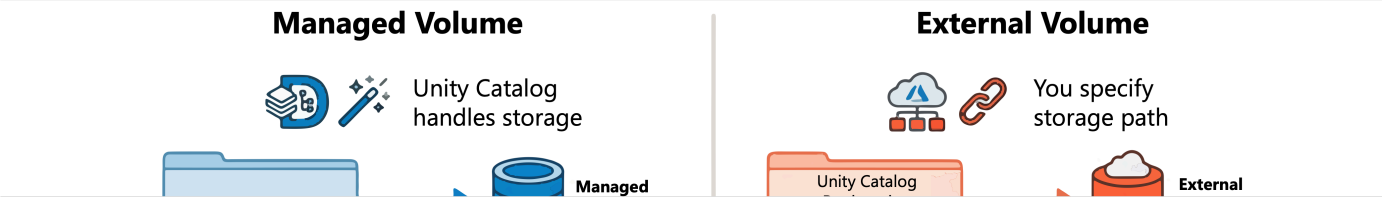
With Unity Catalog's three-layer hierarchy, understanding where volumes fit becomes essential. Volumes handle file-based data governance, providing path-based access to files while maintaining centralized security controls. This separation allows you to manage structured data through tables and unstructured data through volumes, both under the same governance framework.

## Choose between managed and external volumes

When you create a volume, you choose between **managed** and **external** volumes. This decision affects where your files are stored and what happens when you delete the volume.

**Managed volumes** offer the simplest approach. Unity Catalog handles storage location and lifecycle management automatically. When you create a managed volume, Azure Databricks stores files in the managed storage location associated with your schema. When you drop a managed volume, Unity Catalog marks the files for deletion after a seven-day retention period.

**External volumes** provide governance for existing cloud storage locations. You specify the storage path when creating the volume, and the files remain in that location throughout the volume's lifecycle. When you drop an external volume, the files remain in cloud storage—only the Unity Catalog registration is removed.




Consider managed volumes when you work exclusively within Azure Databricks and want simplified storage management. Your data engineering pipelines can write files directly to volumes without worrying about underlying storage paths or credentials.

Consider external volumes when you need to share data between Azure Databricks and other systems. Perhaps your machine learning models must be accessed by external applications, or you need to add governance to files already produced by legacy systems. External volumes allow you to maintain existing storage locations while adding Unity Catalog's security controls.

The technical implementation remains nearly identical regardless of your choice. You access files using the same path format, apply the same permissions, and use the same tools. The main difference lies in storage control and data lifecycle management.

## Create a volume

Creating a volume establishes a governed storage location for your files. You must have **CREATE VOLUME** privilege on the schema and **USE** privileges on both the schema and catalog. For external volumes, you also need **CREATE EXTERNAL VOLUME** privilege on the target external location.

**Using SQL**, create a managed volume with this syntax:

SQL

```sql
CREATE VOLUME IF NOT EXISTS dev_catalog.bronze_schema.landing_files
COMMENT 'Landing area for CSV files from external systems';
```

This creates a volume named `landing_files` in the specified schema. The `IF NOT EXISTS` clause prevents errors if the volume already exists, making your code resilient to repeated execution.

**For external volumes**, add the `LOCATION` clause to specify the cloud storage path:

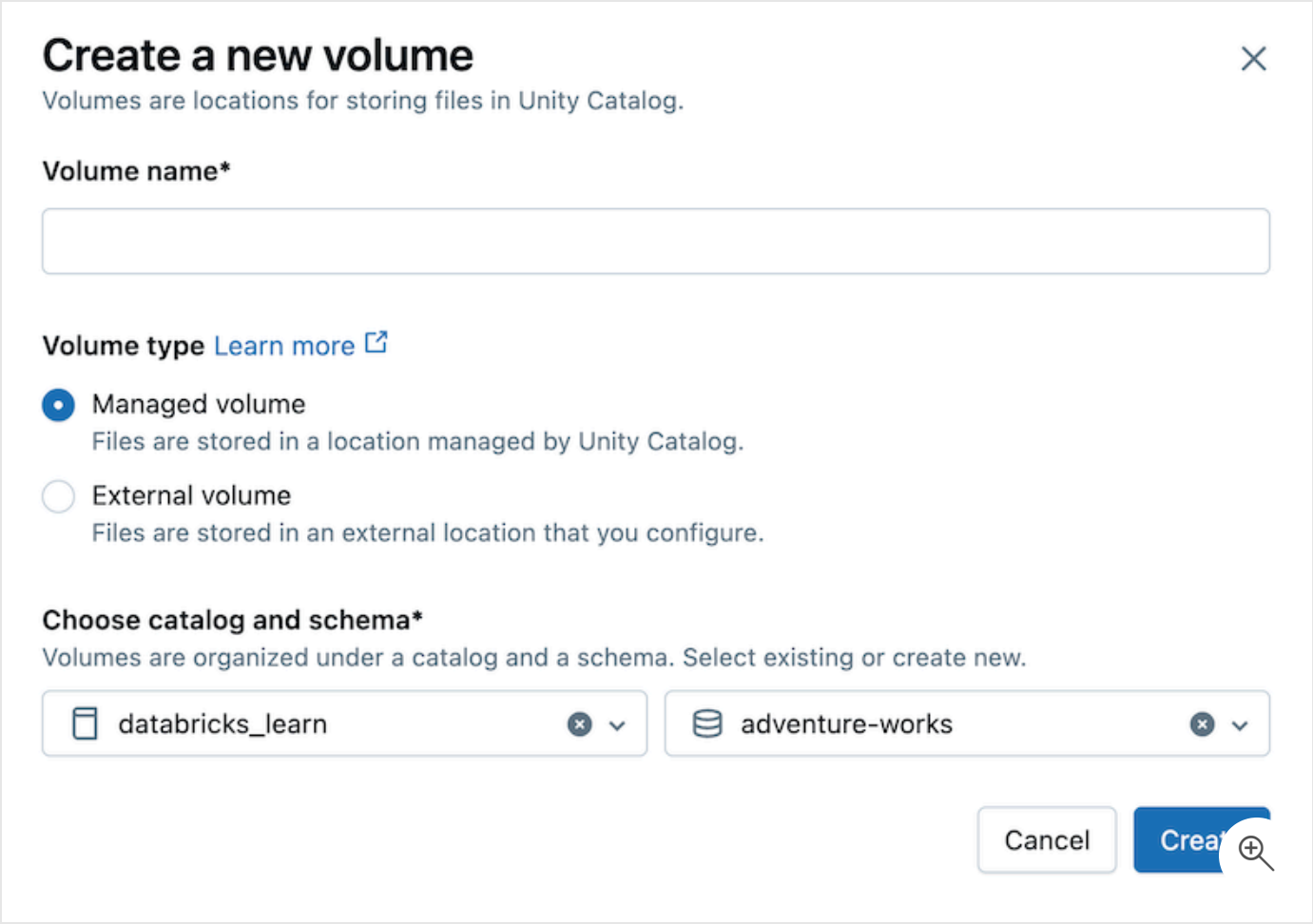
SQL

```sql
CREATE EXTERNAL VOLUME prod_catalog.silver_schema.ml_models
LOCATION 'abfss://models@mystorageaccount.dfs.core.windows.net/production/';
```

The location must point to a path within an external location that Unity Catalog already governs. This ensures that Unity Catalog maintains security control even when the volume references existing storage.

**Using Catalog Explorer**, create volumes through the UI:

1. Select **Catalog** from the left navigation.
2. Navigate to the target schema and select it.
3. Select **Create** > **Volume**.
4. Enter a volume name.
5. Choose **Managed** or **External** as the volume type.
6. For external volumes, select an external location and specify the subdirectory path.
7. Select **Create**.




After creation, your volume is ready to store files. The volume inherits permissions from its parent schema, though you can grant specific permissions to individual users or groups as needed.

## Access files in volumes

Once you create a volume, you access files using a standard path format that works across all Azure Databricks tools and languages. The path follows this pattern:

```txt
```

```
/Volumes/<catalog>/<schema>/<volume>/<path>/<filename>
```

This POSIX-style path works with Apache Spark, pandas, SQL, and other frameworks without requiring cloud storage credentials or connection strings. For example, to read a CSV file stored in your volume:

Python

```python
df = spark.read.format("csv").load("/Volumes/dev_catalog/bronze_schema/landing_files/data.csv")
```

The same path works in pandas:

Python

```python
import pandas as pd
df = pd.read_csv('/Volumes/dev_catalog/bronze_schema/landing_files/data.csv')
```

You can also query files directly using SQL:

SQL

```sql
SELECT * FROM csv.`/Volumes/dev_catalog/bronze_schema/landing_files/data.csv`;
```

For file management operations, use `dbutils.fs` commands in notebooks:

Python

```python
# List files in a volume
dbutils.fs.ls("/Volumes/dev_catalog/bronze_schema/landing_files")

# Copy a file
dbutils.fs.cp(
    "/Volumes/dev_catalog/bronze_schema/landing_files/source.csv",
    "/Volumes/dev_catalog/silver_schema/processed/destination.csv"
)
```

External volumes support an additional access pattern. Users with appropriate permissions can access files using cloud storage URIs directly, such as `abfss://container@account.dfs.core.windows.net/path/file.csv`. However, Unity Catalog still governs this access through volume permissions, not external location permissions.

# Manage volume permissions

Volume permissions control who can read and write files. Unity Catalog provides four key privileges for volumes:

- **READ VOLUME**: Allows listing and reading files in the volume
- **WRITE VOLUME**: Allows creating, updating, and deleting files
- **CREATE VOLUME**: Allows creating new volumes in a schema
- **CREATE EXTERNAL VOLUME**: Allows creating external volumes using an external location

To grant read access to a data engineers group:

SQL

```
GRANT READ VOLUME ON VOLUME dev_catalog.bronze_schema.landing_files TO `data-engineers`;
```

To grant full read and write access to data engineers:

SQL

```
GRANT READ VOLUME, WRITE VOLUME ON VOLUME dev_catalog.bronze_schema.landing_-files TO `data-engineers`;
```

Permissions cascade from parent objects. If you grant READ VOLUME at the catalog level, users can read files from all volumes in that catalog. This makes it simple to set up broad access policies while restricting sensitive data through more specific grants.

> **Note**
>
> Unity Catalog volumes don't support folder-level or subdirectory-level ACLs. Permissions apply to the entire volume. If you need different access controls for different sets of files, create separate volumes for each security boundary.

Remember that reading or writing files requires both volume permissions and the necessary `USE` privileges on the parent catalog and schema. Unity Catalog enforces this hierarchy to maintain consistent access control across your data assets.